

LC 1368: Minimum Cost to Make Atleast One Valid Path in a Grid

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 24, 2026

Morgan Goldman Visa Deloitte

Problem Understanding & Grid Convention

1	2
3	4

(Convention)



→	←
↓	↑

end

- need to go to end, one change costs **1**, you can only change one cell once
- here cost = 1 by →

→	↓
↓	↓

← **just change this**
this cell sign doesn't matter

Key Observations

- If you decide to follow the arrow, then **0 cost**
- decide to change arrow **1 cost**
- Every move has only 2 possible costs
- So every edge has a weight of **0 or 1**
- whenever graph edges are 0 or 1 we don't need priority queue, but **dequeue**

0-1 BFS Algorithm

- `dist[row][col]` for min modif cost
- if a path is as good as current one, process it immediately → **push_front**
- if a path is worse, process later → **push_back**


```

class Solution {
public:
    int minCost(vector<vector<int>>& grid) {

        int n = grid.size();
        int m = grid[0].size();
        vector<pair<int,int>> dir = {{-1,0}, {0,-1}, {1,0}, {0,1}};

        // since edge weight are only 0 and 1 we can use deque
        // to store {cost, {row, col}}
        deque<pair<int, pair<int,int>>> dq;
        vector<vector<int>> dist(n, vector<int>(m, INT_MAX));

        dq.push_front({0, {0,0}});
        dist[0][0] = 0;

        while (!dq.empty()) {
            int r = dq.front().second.first;
            int c = dq.front().second.second;
            int currcost = dq.front().first;
            dq.pop_front();

            if (currcost > dist[r][c]) continue;
            if (r == n-1 && c == m-1) return currcost;

            for (auto &it : dir) {
                int nr = r + it.first;
                int nc = c + it.second;

                if (nr >= 0 && nc >= 0 && nr < n && nc < m) {
                    int newcost = currcost;

                    if (it.first == 0 && it.second == 1) {
                        // right jaana tha but currentcell jaane nhi de rha , to uska direct
                        if (grid[r][c] != 1) {
                            newcost++;
                        }
                    }
                    else if (it.first == 0 && it.second == -1) {
                        // left jaana tha but currentcell jaane nhi de rha , to uska direct
                        if (grid[r][c] != 2) {
                            newcost++;
                        }
                    }
                    else if (it.first == 1 && it.second == 0) {
                        // neeche jaana tha but currentcell jaane nhi de rha , to uska direct
                        if (grid[r][c] != 3) {
                            newcost++;
                        }
                    }
                    else if (it.first == -1 && it.second == 0) {
                        // upar jaana tha but currentcell jaane nhi de rha , to uska direct

```

```

        if (grid[r][c] != 4) {
            newcost++;
        }
    }

    if (newcost < dist[nr][nc]) {
        dist[nr][nc] = newcost;

        if (newcost == currcost) {
            // matlab arrow sign follow kiya
            // hence as good as currpath , so push front
            dq.push_front({newcost, {nr, nc}});
        }
        else {
            dq.push_back({newcost, {nr, nc}});
        }
    }
}

return -1;
};

```